

Name Driven Development

Enumeration as Architecture

Registry: [\[HOWL-COMP-11-2026\]](#)

Series Path: [\[HOWL-COMP-1-2026\]](#) → [\[HOWL-COMP-2-2026\]](#) → [\[HOWL-COMP-3-2026\]](#) → [\[HOWL-COMP-4-2026\]](#) → [\[HOWL-COMP-5-2026\]](#) → [\[HOWL-COMP-6-2026\]](#) → [\[HOWL-COMP-7-2026\]](#) → [\[HOWL-COMP-8-2026\]](#) → [\[HOWL-COMP-9-2026\]](#) → [\[HOWL-COMP-10-2026\]](#) → [\[HOWL-COMP-11-2026\]](#)

DOI: 10.5281/zenodo.20614224

Date: June 2026

Domain: Software Architecture / Systems Engineering

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet.

1. What Name Driven Development Is

Name Driven Development is a software development method where the first act of engineering is exhaustively naming every state change the system can undergo. Before any code is written, before any architecture is chosen, the developer produces a complete flat list of names — one for every distinct thing that can happen in the system. This list becomes the specification, the architecture, the task list, and the documentation simultaneously. Implementation is then the mechanical process of writing the action behind each name.

The method rests on a single claim: if you can name everything that happens in a system, you can build it. Conversely, if you cannot name what happens, you do not yet understand what you are building.

The canonical representation is an enumeration — a flat, ordered, grouped list of names. Not a class hierarchy, not a flowchart, not a box diagram. A list. The list is the architecture.

2. The Naming Act

Naming is the primary intellectual labor of this method. Giving a state change a name requires understanding what triggers it, what it affects, what must have happened before it, and what can happen after it. A name that cannot be explained in one sentence is a name concealing two distinct things and must be split. A sequence that feels incomplete has a missing name — a state change happening implicitly that no one has articulated yet.

Consider a system where a user logs in. The naive approach produces one name: `User_Logged_In`. But that is hiding several things. The user submitted credentials. The credentials were validated. A session was created. The user's environment was loaded. That is four names: `User_Credentials_Entered`, `User_Authenticated`, `User_Session_Created`, `User_Environment_Loaded`. Each one does exactly one thing. Each one can succeed or fail independently. Each one can be implemented, tested, and debugged in isolation.

The discipline is absolute: no code is written until every name exists. This feels slow at the beginning. It is the opposite. The time spent naming is time not spent refactoring, not spent discovering missing states during integration, not spent debugging implicit transitions that nobody documented. The naming phase is where the mistakes are cheapest.

Names are grouped by domain and ordered within each group by lifecycle sequence. Groups begin at round offsets — 100, 200, 300 — leaving room for future additions without renumbering. This is a practical convention, not a theoretical requirement. It keeps the list readable as it grows.

3. The Carrier

A name without context is not actionable. `Init_Filesystems_Mounted` needs to know which filesystem and where. `Services_SSH_Started` needs to know on which port. Every name travels with a small data packet — the carrier.

The carrier is a single struct type shared across all names. It is not polymorphic. It is not specialized per name. It contains a fixed set of optional fields — source, target, a few parameter slots, a timestamp. Any field not relevant to a given name stays at its default value.

A carrier for the OS boot process might look like this:

`BootEvent:`

<code>event: EventType</code>	— which name fired
<code>source_id: integer</code>	— subsystem that caused this
<code>target_id: integer</code>	— subsystem affected
<code>device: text</code>	— device path, if relevant
<code>mount_point: text</code>	— filesystem target, if relevant
<code>status_code: integer</code>	— result, if relevant
<code>elapsed_ms: integer</code>	— time since boot

Most events use two or three of these fields. The rest stay empty. This is intentional. A universal carrier means the event processor has one code path, one logging format, one serialization shape. The name carries semantic meaning. The carrier carries parameters.

The temptation to create specialized carriers per event category — one struct for filesystem events, another for network events, another for service events — must be resisted. The moment the carrier becomes polymorphic, you have reintroduced the type dispatch complexity that naming was meant to eliminate. One carrier. One shape. Optional fields for what varies.

4. The Action

Each name maps to one action. The action is the simplest code that does what the name says. Nothing more.

`Init_Swap_Enabled` enables swap. It reads the carrier's device field, calls the system call, updates the state. Five lines. `Services_Logging_Started` starts the logging daemon. It reads a configuration path, spawns the process, records the PID. Ten lines. `Kernel_Root_Filesystem_Mounted` mounts the root filesystem. It reads device, mount point, and filesystem type from the carrier, calls mount, checks the return code. Eight lines.

Actions do not inspect the event name. An action never contains a conditional that asks “which event am I handling?” That question was already answered by the dispatch. If an action needs to branch on the event type, it means two different things are hiding under one name, and the name must be split.

Actions do not call other actions. If `Services_SSH_Started` needs logging, it does not call the action for `Services_Logging_Started`. It emits a new event with that name. The sequence orchestrator handles ordering. Actions are leaves, not nodes in a call graph.

This constraint keeps each action trivially testable. Pass in a carrier with the right fields populated, run the action, check the result. No setup beyond the carrier. No mocks for collaborating handlers. The action's entire world is the name it responds to and the carrier it receives.

5. The Sequence

Individual actions are inert. The program is the sequence in which they fire.

An OS boot is a sequence of roughly sixty names in a fixed order. The orchestrator — whatever drives execution — walks the list. It populates a carrier for each step, fires the name, checks the result, and proceeds to the next name or halts on failure.

Some sequences are linear. The BIOS phase fires names in strict order: `BIOS_Power_On_Self_Test`, then `BIOS_Hardware_Enumerated`, then `BIOS_Boot_Device_Selected`,

then `BIOS_MBR_Loaded`, then `BIOS_Bootloader_Transferred`. No branching. Each name fires once.

Some sequences branch. After `User_Credentials_Entered`, the next name is either `User_Authenticated` or the credentials are rejected and the sequence returns to the login screen. The branching logic lives in the orchestrator, not in the action. The action for `User_Credentials_Entered` validates credentials and returns a result. The orchestrator reads that result and decides the next name.

Some sequences run concurrently. Service startup often parallelizes — `Services_SSH_Started` and `Services_Cron_Started` and `Services_Dbus_Started` have no ordering dependency and can fire simultaneously. The orchestrator knows this because the dependency graph between names is explicit and declared, not implicit in call order.

The critical property is that all conditional logic, all branching, all ordering decisions live in the orchestrator. Actions never decide what happens next. They execute and return. The orchestrator is the only component that knows the shape of the program. Everything else knows only its own name.

6. How This Differs From Event-Driven Architecture

Name Driven Development uses events. Event-driven architecture uses events. They appear similar. The difference is fundamental and produces radically different software.

When events are defined.

In event-driven architecture, events are defined during implementation. A developer building the payment module realizes the order module needs to know when payment succeeds, so they create a `PaymentCompleted` event. Events emerge bottom-up from the needs of communicating components. The full set of events is never enumerated in advance — it is the accumulated residue of implementation decisions.

In Name Driven Development, all events are defined before implementation begins. The developer names every state change in the payment system before writing any code: `Payment_Initiated`, `Payment_Authorized`, `Payment_Captured`, `Payment_Failed`, `Payment_Refund_Requested`, `Payment_Refunded`. The full set exists first. Implementation fills in the actions.

Where logic lives.

In event-driven architecture, handlers contain business logic. An `on_payment_completed` handler might update inventory, generate an invoice, notify shipping, send a confirmation email, and log an audit entry. The handler is the locus of complexity. Understanding what happens when payment completes requires reading that handler and every handler it triggers transitively.

In Name Driven Development, the equivalent is five separate names: `Inventory_Updated`, `Invoice_Generated`, `Shipping_Notified`, `Con-`

`firmation_Sent`, `Audit_Logged`. Each does one thing. The orchestrator fires all five in sequence after `Payment_Captured`. No single action contains the compound logic. Understanding what happens after payment requires reading the sequence, which is a list of names — not code.

How you know when you are done.

In event-driven architecture, completeness is tested by running the system and observing whether the correct behaviors occur. There is no static artifact that tells you whether all cases are handled. A missing handler is a runtime bug, discovered through testing or in production.

In Name Driven Development, completeness is visible by inspection. The enum is the checklist. Every name with an implemented action is done. Every name without one is not. You can count remaining work by counting unimplemented names. This property alone — visible completeness — changes how software is planned, estimated, and tracked.

How the design evolves.

In event-driven architecture, adding a feature means adding handlers, possibly creating new events, and wiring them into the existing event graph. The impact of a new handler on the existing system requires tracing all event flows that might intersect with it.

In Name Driven Development, adding a feature means adding names to the enum. The new names are visible in one place. Their position in the list shows which domain they belong to. Their implementation is isolated. Their sequencing is explicit in the orchestrator. The blast radius of a change is bounded by the names involved.

7. The Spec-to-Names Pipeline

The input to Name Driven Development is a natural language specification. The spec describes what the system does — every behavior, every interaction, every rule. It is written in domain language, not technical language. It is written for completeness of thought, not for code structure.

Extracting names from the spec is a decomposition task. Every verb-noun pair is a candidate name. “The kernel mounts the root filesystem” yields `Kernel_Root_Filesystem_Mounted`. “Services start in dependency order” yields a name per service: `Services_Logging_Started`, `Services_Dbus_Started`, `Services_SSH_Started`. “The user enters credentials and is authenticated” yields two names, not one, because entering and validating are distinct state changes.

The extraction process can be manual — a developer reading the spec with a text editor open, pulling out names one by one. It can be collaborative — a team at a whiteboard decomposing the spec section by section. It can be accelerated with a large language model — feeding the spec as input and asking for every distinct state change as output.

The LLM is well suited to this step because it is pattern extraction, not design. The spec already contains the events implicitly in its prose. The LLM reads sentences like “suppressed demand accumulates and releases as a surge when suppression ends” and produces `Suppression_Demand_Accumulated`, `Suppression_Released`, `Suppression_Surge_Triggered`. It is not inventing — it is decomposing. The domain vocabulary comes from the spec, so naming consistency comes for free.

Regardless of extraction method, the output is a draft. The developer reads the draft critically. Names that cannot be explained in one sentence are split. Names that always fire together are merged. Gaps in the sequence are filled. The developer’s judgment is the quality gate. The extraction method is the accelerator.

8. Refinement and the Living Enum

The enum is never frozen. It is a living document that evolves with the system.

During implementation, names are discovered to be missing. A developer implementing `Kernel_Modules_Loaded` realizes that module loading can fail, and there is no name for that. `Kernel_Module_Load_Failed` is added to the enum. The action is written. The orchestrator is updated to handle the branch.

Names are discovered to be redundant. Two names that always fire in immediate succession with identical carriers are collapsed into one. The enum shrinks. The implementation simplifies.

Names are discovered to need splitting. `Services_Network_Started` turns out to cover both interface configuration and DNS resolver setup, which can fail independently. It becomes `Init_Network_Interfaces_Configured` and `Init_DNS_Resolver_Configured`. The enum grows. The implementation becomes more precise.

Adding a feature begins with adding names. Removing a feature begins with removing names. Refactoring is renaming. The enum is the changelog. A diff of the enum between two versions of the software tells you exactly what changed in terms of capability — not in terms of code, but in terms of what the system can do.

Progress tracking is counting. The enum has 60 names. 45 have implemented actions. 15 remain. The project is 75% complete, and this number is meaningful because each name represents a unit of functionality, not a unit of code. Some actions are five lines. Some are fifty. But each one is a complete, testable, deployable behavior.

9. Worked Example: The OS Boot Process

The specification in natural language: a computer powers on. Hardware initializes itself and selects a boot device. A bootloader loads the operating system kernel into memory. The kernel initializes memory management, discovers hardware, and mounts the root

filesystem. An init process starts system services in dependency order. A display server launches and presents a login screen. The user authenticates and arrives at a working desktop.

Seven sentences. From these, approximately sixty names are extracted, organized into seven groups.

BIOS (offsets 100–199). BIOS_Power_On_Self_Test, BIOS_Hardware_Enumerated, BIOS_Boot_Device_Selected, BIOS_MBR_Loaded, BIOS_Bootloader_Transferred. Five names. The entire firmware phase. Each one is a known, bounded task with clear success and failure conditions.

Bootloader (200–299). Bootloader_Stage1_Loaded, Bootloader_Stage2_Loaded, Bootloader_Kernel_Located, Bootloader_Kernel_Loaded_To_Memory, Bootloader_Initrd_Loaded, Bootloader_Kernel_Parameters_Set, Bootloader_Control_Transferred. Seven names. The transition from firmware to operating system.

Kernel (300–399). Kernel_Entered, Kernel_Page_Tables_Initialized, Kernel_Memory_Manager_Started, Kernel_Interrupt_Table_Built, Kernel_Timer_Started, Kernel_Console_Initialized, Kernel_PCI_Bus_Enumerated, Kernel_Block_Devices_Discovered, Kernel_Root_Filesystem_Mounted, Kernel_Initrd_Unpacked, Kernel_Modules_Loaded, Kernel_Root_Switched. Twelve names. The kernel bootstrap — the most complex single phase, yet each name is still one action.

Init (400–499). Init_Process_Started, Init_Runlevel_Determined, Init_Hostname_Set, Init_Sysctl_Applied, Init_Udev_Started, Init_Devices_Populated, Init_Filesystems_Checked, Init_Filesystems_Mounted, Init_Swap_Enabled, Init_Clock_Synchronized, Init_Loopback_Interface_Up, Init_Network_Interfaces_Configured, Init_Firewall_Rules_Applied, Init_DNS_Resolver_Configured. Fourteen names. System preparation before services.

Services (500–599). Services_Logging_Started, Services_Dbus_Started, Services_Cron_Started, Services_SSH_Started, Services_Audio_Initialized, Services_Bluetooth_Started, Services_Print_Spooler_Started, Services_Network_Manager_Started, Services_VPN_Connected. Nine names. Each service is one name. Adding a new service to the system means adding one name, writing one action, inserting it into the sequence.

Display (600–699). Display_GPU_Driver_Loaded, Display_Framebuffer_Initialized, Display_Server_Started, Display_Session_Manager_Started, Display_Login_Screen_Rendered. Five names. The graphical stack.

User (700–799). User_Credentials_Entered, User_Authenticated, User_Session_Created, User_Environment_Loaded, User_Autostart_Applications_Launched, User_Desktop_Rendered, User_Input_Ready. Seven names. From login prompt to working desktop.

Reading this list from top to bottom, a reader who has never seen the source code understands the boot process. They know what happens, in what order, and where the boundaries are between subsystems. They know that if SSH fails to start, the names after it in the Services group still fire — services are independent. They know that if `Kernel_Root_Fileystem_Mounted` fails, nothing in the Init group can proceed — it is a hard dependency.

This is the property that distinguishes Name Driven Development from documentation written after the fact. The names are not a description of the software. They are the software's skeleton. The implementation is the muscle attached to that skeleton. You cannot have the software without the names, and the names without the software are a complete specification of what needs to be built.

10. When To Use This

Name Driven Development works when the system has a finite, enumerable set of state changes. This is most software.

It works exceptionally well for systems with long lifecycle flows — boot processes, order fulfillment, claims processing, manufacturing pipelines. The flow is the sequence of names. Understanding the flow means reading the list.

It works exceptionally well for systems with many interacting subsystems — operating systems, games, enterprise platforms, IoT device networks. Each subsystem is a group of names. The boundaries between subsystems are the boundaries between groups. Integration is sequencing names across groups.

It works exceptionally well for systems where completeness matters — medical records, financial transactions, regulatory compliance. The enum is an auditable artifact. Every state change the system can undergo is listed. A compliance reviewer can read the names and verify that every required transition is present.

It works less naturally for pure data transformation — compilers, image processors, numerical simulations. These systems are dominated by algorithms operating on data, not by discrete state changes. The interesting work is in the math, not in the transitions between states. Names can still describe the pipeline stages, but the value of enumeration is lower when the complexity lives inside an action rather than in the sequencing of actions.

It works at any scale. A script with ten names is a well-organized script. A platform with two thousand names is a comprehensible platform — two thousand lines in a single file, grouped and ordered, readable in an afternoon. The same cannot be said of two thousand handlers scattered across two hundred source files.

The method does not prescribe a language, a framework, or a runtime. The enum can be a Zig enum, a Python class with constants, a TypeScript union, a database table of event codes, or a column in a spreadsheet. The carrier can be a struct, a dictionary, a JSON object, or a row in a message queue. The orchestrator can be a loop, a state machine, a

workflow engine, or a human clicking buttons. The method is the discipline of naming first and coding second. Everything else is implementation detail.

HOWL-COMP-11-2026. Name Driven Development: Enumeration as Architecture.

[[HOWL-COMP-11-2026](#)] Name Driven Development. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-11-2026>

[[HOWL-COMP-1-2026](#)] Implementing a Tall-Infra Data-Only Execution System .
Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-1-2026>

[[HOWL-COMP-2-2026](#)] Tall-Infra, Data-Only Development. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-2-2026>

[[HOWL-COMP-3-2026](#)] Silo OS. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-3-2026>

[[HOWL-COMP-4-2026](#)] Geometric Security. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-4-2026>

[[HOWL-COMP-5-2026](#)] Partial Geometric Security. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-5-2026>

[[HOWL-COMP-6-2026](#)] SIML - Silo Markup Language (TOML Format). Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-6-2026>

[[HOWL-COMP-7-2026](#)] SiQL. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-7-2026>

[[HOWL-COMP-8-2026](#)] SiloSec Protocol Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-8-2026>

[[HOWL-COMP-9-2026](#)] Building Applications with OpsDB Application Architecture.
Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-9-2026>

[[HOWL-COMP-10-2026](#)] Tall-Infra Data-Only Execution. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-10-2026>